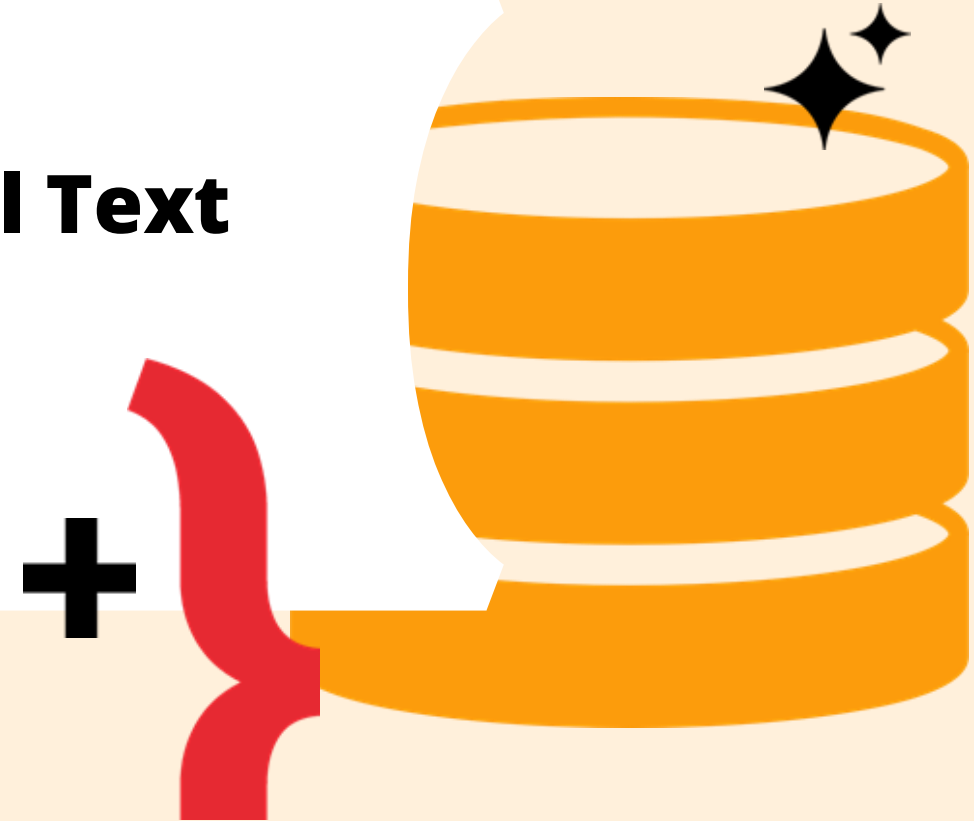
Couchbase

citi

Introduction to Full Text and Vector Search



Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Agenda

Full Text Search

Demystifying Vector Search

Use Case: Semantic Search

Use Case: Retrieval-Augmented Generation

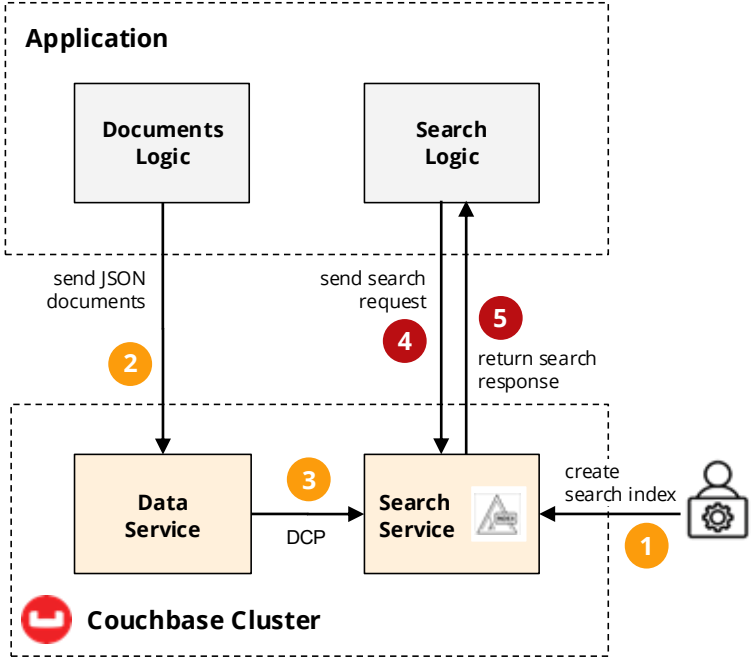
Key Takeaways

Full Text Search Overview

Overview | Store, Index & Query documents with Couchbase

The Data Service is the source of truth, while the Search Service provides both Indexing and Querying

1. A search index is created in the Couchbase Search service
2. Application sends business documents as JSON documents to the Data Service (or KV).
3. The Search Service responds to document mutations via DCP. The search index in Search is maintained in near real-time.
4. A query is sent from the Application to the Search service.
5. The Search service responds back to the Application with the most relevant documents (TF-IDF/BM25).



Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Overview | When to use Search service instead of GSI with SQL++

Use GSI

(WHERE type = "shoes" and size = 6 AND price between 49.99 and 99.99)

GSI works great for range searches and string searches when data is organized.

Use GSI

(WHERE lower(name) LIKE "joe%" AND state = "CA")

The GSI indexes perform an exact search in ms. Predicates of patterns (name LIKE "%turin%") will scan the whole index which is unsuitable for application use.

Use Search Service

Find joe in any field within the JSON document

GSI based indexes suffer when you look for the word fragment "joe" or match phone numbers with various patterns in a larger document.

Use Search Service

Find documents matching phone number (408-956-2444) in any format (+1 (408) 956-2444, +1 510.956.2444, (408) 956 2444)

Use FTS when you need to use filters on multiple array fields in a single query. FTS is efficient in indexing and query arrays.

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Overview | Couchbase Search Indexing and Queries

INDEXING

- Real time indexing
- Default map and map by document type
- Dynamic mapping
- Stored fields
- Term vectors
- Analyzers
- Synonyms

QUERIES

- Basic
- Compound
- Range
- Vector
- Special Purpose
- Scoring (TF/IDF & BM25)
- Boosting

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Search Queries

Since Couchbase Server 7.6, you can perform Vector Queries for similarity search, semantic search and generative AI

- **Simple Queries**
- **Compound Queries**
- **Range Queries** (string, date, numeric)
- **String Queries** (natural language)
- **Geospatial Queries**
- **Exact match Queries**
- **Special Queries** (for dev purpose)
- **Vector Queries** (similarity search, semantic search, generative AI) — Since 7.6

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Search Queries | Interacting with Search

You can perform Search Queries through REST API, Couchbase SDKs or SQL++

REST API

```
{
   "match": "location hostel",
   "field": "reviews.content",
   "analyzer": "standard",
   "fuzziness": 2,
   "prefix_length": 4,
   "operator": "and"
}
```

- See How-to Documentation

SDK CLIENTS

```
public static void simpleTextQuery(Bucket bucket)
{
        String indexName = "travel-sample-index";
        MatchQuery query =
SearchQuery.match("swanky");
        SearchQueryResult result = bucket.query(new
                SearchQuery(indexName,
query).limit(10));
        printResult("Simple Text Query", result);
}
```

- Currently available in all Couchbase SDKs
- See How-to Documentation

Through SQL++

```
SELECT t1.name
FROM `travel-sample` AS t1
WHERE SEARCH(t1, {
      "match": "bathrobes",
      "field": "reviews.content",
      "analyzer": "standard"
});
```

- Through the Search function
- See SQL++ Documentation

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Search Queries | Simple Queries

Query Type	Example	Matched Terms Results
MATCH	{"field": "reviews.content", **"match"**: "beautiful"}	beauty beautiful
MATCH PHRASE	{"field": "reviews.content", "**match_phrase**": "beautiful location"}	beautifully located beautiful location
FUZZY	{"field": "reviews.content", "**match**": "hotel", "**fuzziness**":"1"}	hotel hostel
PREFIX	{field": "reviews.content", "**prefix**":"bea"}	beach beautiful

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Search Queries | Simple Queries *(Cont'd)*

Query Type	Example	Matched Terms Results
REGEXP	{“field": "reviews.content", “**regexp**":”ho[st|t]el”}	hostel hotel
WILDCARD	{“field": "reviews.content", “**wildcard**”:”ho?tel”}	hostel
BOOLEAN FIELD	{ “field": "reviews.content", "**bool**": true, "**field**":"free_breakfast"}	documents where the field contains boolean true value

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Search Queries | Fuzziness

Fuzziness allows you to find words when the search term is misspelled.

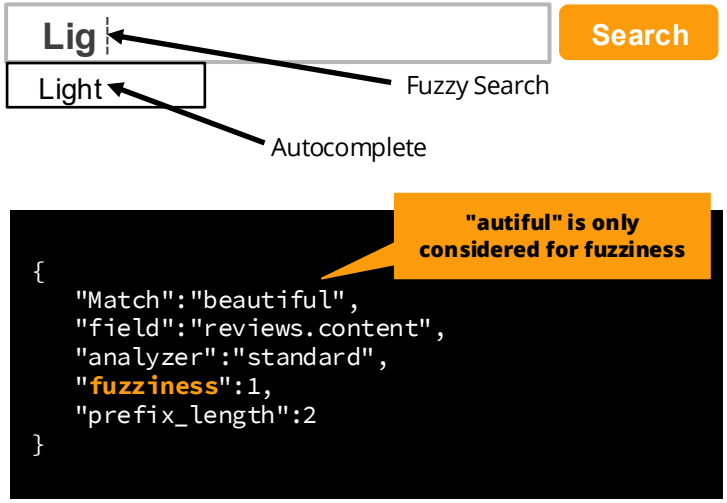
Overview

- The `fuzziness` parameter indicates how many letters may be different or missing, leveraging the Levenshtein distance.
- Maximum supported fuzziness is 2 to lower the number of false positives

Optimization

- Most spelling mistakes happen towards the end
  => utilize the `prefix_length` option in fuzzy queries

Fuzziness and Autocomplete



Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Search Queries | Compound Queries

Query Type	Description	Example
CONJUNCTION	Logical AND. Contains multiple child queries. Its result documents must satisfy all of the child queries.	{ "**conjuncts**": [{"field": "reviews.content", "match": "location"}, {"field": "free_breakfast", "bool": true}]}
DISJUNCTION	Logical OR. Contains multiple child queries. Its result documents must satisfy a configurable min number of child queries. By default this min is set to 1.	{"**disjuncts**": [{"field": "reviews.content", "match": "location"}, {"field": "free_breakfast", "bool": true}]}

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Search Queries | Compound Queries *(Cont'd)*

Query Type	Description	Example

BOOLEAN	Combination of conjunction and disjunction queries and takes three lists of queries:
	• **must:** Result documents must satisfy all of these queries.
	• **should**: Result documents should satisfy these queries.
	• **must not**: Result documents must not satisfy any of these queries.
		{"**must**": {
		"conjuncts": [
		{"field": "reviews.content",
		"match": "location"}]},
		"**must_not**": {
		"disjuncts": [
		{"field": "free_breakfast",
		"bool": false}]},
		"**should**": {
		"disjuncts": [
		{"field": "free_breakfast",
		"bool": true}]}}

DOC ID	Returns the indexed document or doc among the specified set. This is typically used in conjunction queries, to restrict the scope of other queries' output.	{ "**ids**": ["hotel_10158", "hotel_10159"] }

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Search Queries | Boosting

Boosting allows you to give different weights to each element in a compound query.

Overview

- The boost parameter in Couchbase Search is used to increase or decrease the relative importance (weight) of a clause, field, or subquery when calculating a document's relevance score.
- In simple terms, it amplifies the influence of certain terms or fields on the final ranking.

Example

- Performs Match Queries for `pool` in both the `reviews` and `description` fields, but documents having the term in the `reviews` field score higher.
- Typical values:
  Default → 1
  Increases weight → >1.0 [example: 2, 3]
  Decreases weight → <1.0 [example: 0.5]
  Range: Any positive floating point number

Boosting Example

```
{
  "conjuncts": [
    {
      "field": "description",
      "match": "pool"
      },
    {
      "field": "reviews",
      "match":"pool",
      "boost":2
      }
  ]
}
```

Docs having the term in the reviews field score higher.

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Search Queries | Range Queries

Query Type	Description	Example

DATE RANGE	Finds documents containing a date value, in the specified field within the specified range.	{
		"**start**": "2001-10-09T10:20:30-08:00",
		"**end**": "2016-10-31",
		"**inclusive_start**": false,
		"**inclusive_end**": false,
		"field": "review_date"
		}

NUMERIC RANGE	Finds documents containing a numeric value in the specified field within the specified range	{
		"**min**": 100,
		"**max**": 1000,
		"**inclusive_min**": false,
		"**inclusive_max**": false,
		"**field**": "id"
		}

TERM RANGE	Finds documents containing a term in the specified field within the specified range.	{
		"**min**": "foo",
		"**max**": "foof",
		"**inclusive_min**": false,
		"**inclusive_max**": false,
		"**field**": "desc"
		}

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Search Queries | GeoSpatial Query

Query Type	Description	Example
POINT DISTANCE		{“field”:”geo”, “**location**”:”53.482358,-2.235143”, “**kilometers**”,”1” }
BOUNDED RECTANGLE		{“field”:”geo”, "**top_left**": [-2.235143, 53.482358], "**bottom_right**": [28.955043, 40.991862], }

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Search Queries | GeoSpatial Query *(Cont'd)*

Query Type	Description	Example
BOUNDED POLYGON		{"field": "geo", "**polygon_points**": ["37.79393211306212,-122.44234633404847", "37.77995881733997,-122.43977141339417", "37.788031092020155,-122.42925715405579", "37.79026946582319,-122.41149020154114", "37.79571192027403,-122.40735054016113"] }

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Search Queries | Vector Search Queries

A Vector Search is defined through the *knn* field of a search query

- **knn**: an array of 1 or more vector queries
- **field**: the field in the documents that contains the vector that is indexed
- **vector**: an actual numeric float array to compare against all the indexed vectors
- **k**: the number of nearest hits to return
- [optional] **boost**: a weighting applied to the results if merged with other vectors or hybrid search queries
- [optional] **knn_operator:** can be "", "or", or "and". Defaulted to "or". Logical operator if searching with multiple vectors in the KNN component.

Example of a Search Query

```
{
  "query": {
      "prefix": "S",
      "field": "city",
      "boost": 1.0
  },
  "knn": [{
      "field": "cityVector",
      "vector": [-0.00810323283,0.0727998167,..., -0.0254271757],
      "k": 3,
      "boost": 0.7
  }],
  "knn_operator": "and",
  "fields": ["name", "city"],
  "sort": ["-_score"]
  "limit": 5
}
```

FTS search side

Vector search side

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Search Queries | Vector Search Queries Examples

You can perform either pure Vector Search or Hybrid Search

Example of Pure Vector Search

```
{
   "knn": [{
       "field": "cityVector",
       "vector": [-0.00810323283,...,-0.0254271757],
       "k": 3,
   }],
   "fields": ["name", "city"],
   "sort": ["-_score"]
   "limit": 5
}
```

- NOTE: Since 7.6.2, you don't need to add **"query": {"match_none": {}}** to indicate a pure vector search. It is implicit.

Example of Hybrid Search

```
{
   "query": {"match_all": {}},
   "knn": [{
       "field": "cityVector",
       "vector": [-0.00810323283,...,-0.0254271757],
       "k": 3,
   }],
   "fields": ["name", "city"],
   "sort": ["-_score"]
   "limit": 5
}
```

- **"query": {"match_all": {}}**: will match everything from the traditional search side and combine the results with the "knn" (or vector search) side.

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Search Queries | When to use Pure Vector Search

	1	2	3
	Similarity Search	**Semantic Search**	**RAG Applications**
Use Case	When you want to find data points that are mathematically similar based on vector distance (e.g., cosine similarity)	When you want to retrieve items based on meaning or intent using embeddings to understand natural language context.	When you want to fetch semantically relevant documents to feed into LLMs or generative AI pipelines.
Example	Given an image of a blue shoes, return the 20 product images with the most similar embeddings (aqua, azure, etc)	A search for “heart attack” returns documents about “myocardial infarction”	Context retrieval for generating personalized summaries

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Search Queries | When to use Hybrid Search

	1	2
	Combine Precise Keyword Filtering with Semantic Ranking	**Improve Search Relevance for Broad or Ambiguous Queries**
Use Case	When you want to restrict the dataset via keyword filters, then rank filtered results using vector similarity for richer relevance.	Search handles specificity; vector search handles context and nuance. Together, they deliver more accurate outcomes.
Example	Find research papers in the “science” category and rank them by how closely their abstract matches the “quantum entanglement”	Searching product catalogs where users use natural language rather than structured terms.

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Search Queries | SQL++ with SEARCH Predicate

- Identify the customer accounts and their related contacts where a particular topic has been discussed.
- The search criteria may include one or many of the following informations: **meeting Title, Date range, Customer Contact Details, Sales team member details**

```
SELECT meta(a).id, a.title,a.startDate, a.account.name, a.contacts, a.participants
FROM crm a
WHERE SEARCH(a,
     {"conjuncts": [
          {"field":"title", "match": "artificial intelligence"}   ,
          {"field":"participants.name", "match":"james"}   ,
          {"field":"account.name", "match":"willis"}  ,
          {"field":"startDate","start": "2019-03-20", "end":"2019-03-31"} ,
          {"field":"contacts.name", "match":"boone"}   ,
          {"field":"contacts.email", "match":"obell@gmail.com"}
        ]
     },
     {"index":"all_acts"}
   )
AND a.type='activity'
AND a.activityType='Appointment'
```

Search Clause

Search Index Tip

SQL CLause

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Search Queries | SQL++ with FLEX Indexing

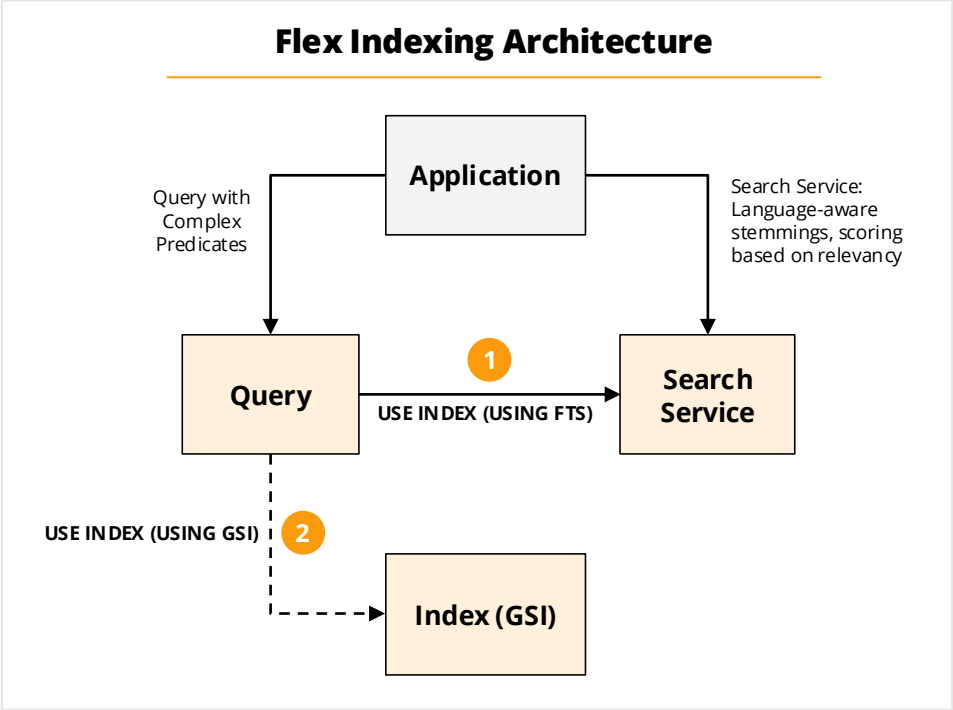
FLEX Index allows the Query to leverage either GSI or FTS

Overview

- Mechanism whereby a SQL++ query can leverage either or both Secondary Index and Keyword Text Search with standard SQL++ predicates

Benefits

- Allows SQL++ to transparently benefit the full power of Text Search capability without any Query limitations
- Examples: Nested document field, Array, SQL Aggregation, Join and Sorting



Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Search Queries | SQL++ with FLEX Indexing

FLEX Index allows the Query to leverage either GSI or FTS

There are two ways to specify that you would like to use a full-text index with a SQL++ query without search predicate:

- Use the **USE FTS** hint in the SQL++ query.
- Set the **use_fts** request-level parameter to **true**.

Example of FLEX Query

```
SELECT META(b).id
FROM mybucket AS b
USE INDEX (USING FTS)
WHERE b.f1 = "xyz" AND b.f2 = 100;
```

Requirements	Description
Search Index	Analysers, Type mappings and Indexed fields must satisfy requirements listed in the documentation.
Query	To use a full-text index with a SQL++ query, the query predicates (conditions and expressions) must also meet requirements listed in the documentation.

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Search Queries | When to use SQL++ with FLEX Indexing

Context	Example
Where the search conditions of the SQL++ statements are not predetermined	It is difficult to create indexes to cover all of the search conditions.
Applications that provide search capabilities involving a large number of predicates	Logical operators, such as AND/OR combinations in the search conditions.
Where the search conditions involve predicates on hierarchical document elements	Such as search that involve array elements in an array, or in multiple arrays.
Where the applications require both the power of Search and need SQL aggregation	With JOIN to include related information from other objects

Or you simply want to use the SQL++ predicate syntax over the Search syntax

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Search Results

Search Results | Sorting

Sorting Type	Description	Example

Sorting with Strings	Array of strings containing either:
	● field name
	● _id
	● _score
		{
		"fields": ["title"],
		"**sort**": ["country", "-_score","-_id"],
		"query":{"query": "beautiful pool"}
		}

Sorting with objects	Fine-grained control over sort-procedure.
	Each object can have the following fields:
	● by
	● field
	● missing
	● Mode
	● type
		{
		"**sort**": [
		"country",
		{ "by" : "field",
		"field" : "reviews.ratings.Overall",
		"mode" : "max",
		"missing" : "last",
		"type": "number"}
		}

- The required sort-type is specified by using the `sort` field, as an array of String or Objects.
- Combination is possible and the default sort-order is ascending.
- In order to change the sort-order to descending, add a negative sign "-" before the sort field as shown above in the example.

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Search Results | Pagination

Pagination Type	Description	Example

SIZE/LIMIT, FROM/OFFSET	• To obtain a subset of results and works deterministically when combined with a sort order
	• Default order is relevance
		{
		"query":
		{ "match": "California",
		"field": "state"},
		"size": 5,
		"**from**": 10
		}

**SEARCH_AFTER
SEARCH_BEFORE**
	• To fetch the size number of results after or before the key specified — for more efficient pagination
	• Allow for the client to maintain state while paginating.
		{
		"query":
		{"match": "California",
		"field": "state"},
		"sort": ["_id"],
		"**search_after**": ["hotel_10180"],
		"size": 3
		}

- Pagination of large number of results are essential for sorting and displaying a subset of these results.

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Search Results | Facets

1

TERM FACET

Use a term facet to return a count for the number of documents in your search results that have a specific term in a field.

2

NUMERIC RANGE FACET

Use a numeric range facet to define a numeric range and receive the number of documents with a field value that falls within that range.

3

DATE RANGE FACET

Use a date range facet to define a date range and receive the number of documents with a date field value that falls within that range.

Facets are aggregate information collected on a particular search result set.
You do a search and collect additional facet information along with it.

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Search Results | Facets (Example)

Term Facet Query

Search documents containing air fresheners
and compute facets on the field type

```
{
    "query": {
       "query": "air fresheners"},
       "facets": {
         "type": {"field": "type"}
    }
}
```

Term Facet Results

Documents containing air fresheners grouped in 2 facets:

- Electric Air Fresheners (288 results)
- Air Fresheners Sprays (887 results)

```
"facets": {
   "type": {
      "field": "type",
        "total": 1175,
        "terms": [
          {"term": "Electric Air Fresheners",
           "count": 288},
          {"term": "Air Freshener Sprays",
           "count": 887}
        ]
    }
}
```

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Concepts

Concepts | Engine behind Search “Bleve”

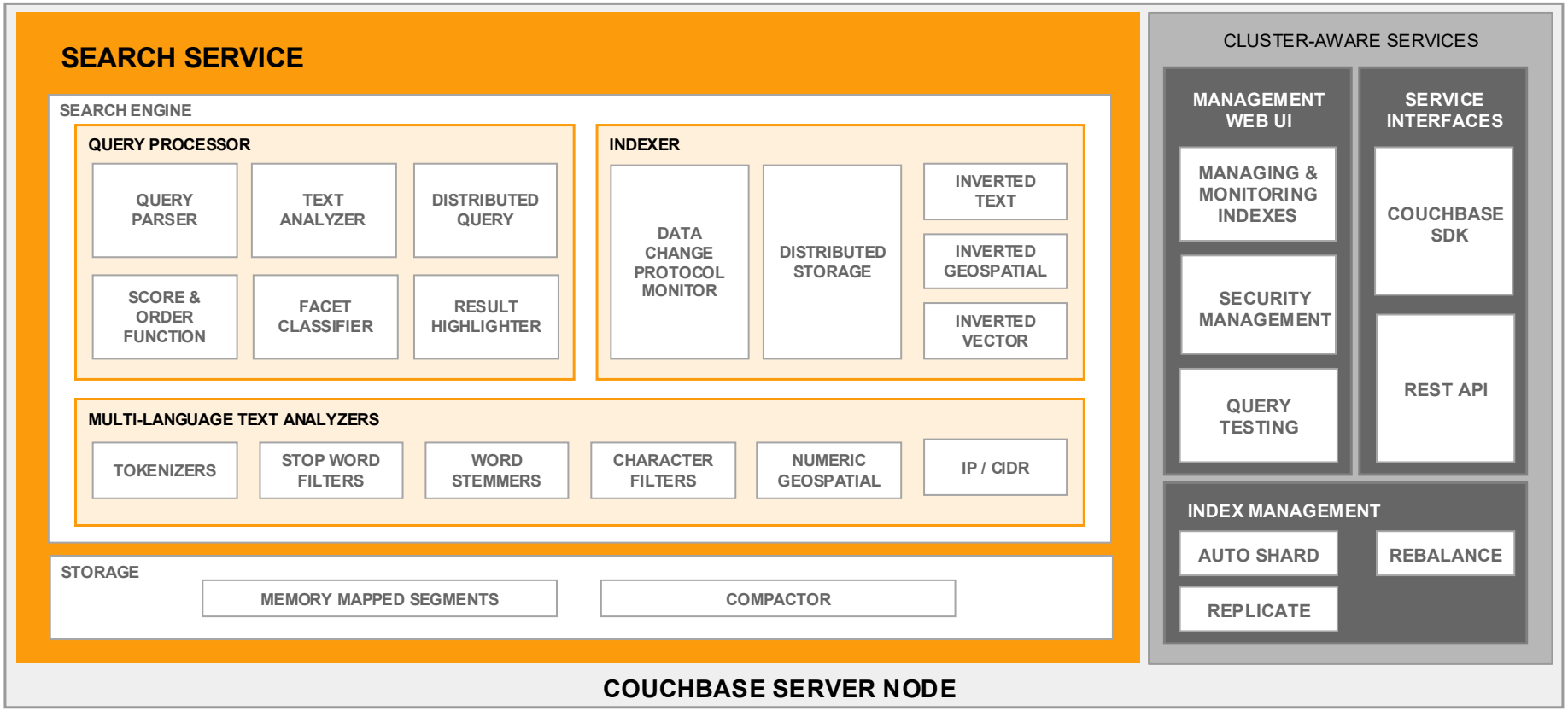
BLEVE — FTS Engine

```go
import "github.com/blevesearch/bleve"/v2

func main() {
    // open a new index
    mapping := bleve.NewIndexMapping()
    index, err := bleve.New("example.bleve", mapping)

    // index some data
    err = index.Index(identifier, your_data)

    // search for some text
    query := bleve.NewMatchQuery("text")
    search := bleve.NewSearchRequest(query)
    searchResults, err := index.Search(search)
}
```

- **BLEVE** = “**B**oiling **L**iquid **E**xpanding **V**apor **E**xplosion”.
- Open source full text search and indexing library.
- Simple, powerful search engine with text analysis, faceting, scoring capabilities (inspired by Lucene).
- Bleve is a text indexing package for Go — no JVM and memory challenges such as ElasticSearch or Apache Solr.
- No need for an extra service + connector/library to get text indexing with scoring, faceting and highlights in your service.

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

Concepts | Search service architecture



Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

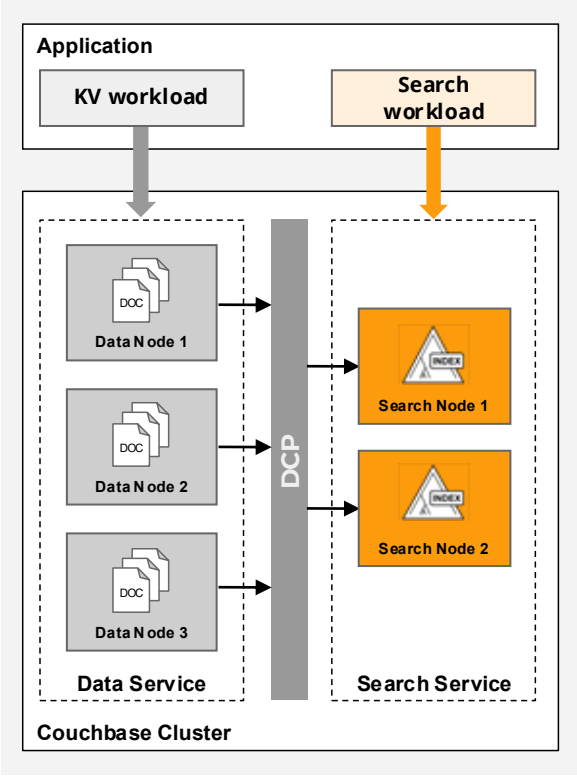
Concepts | Search Features

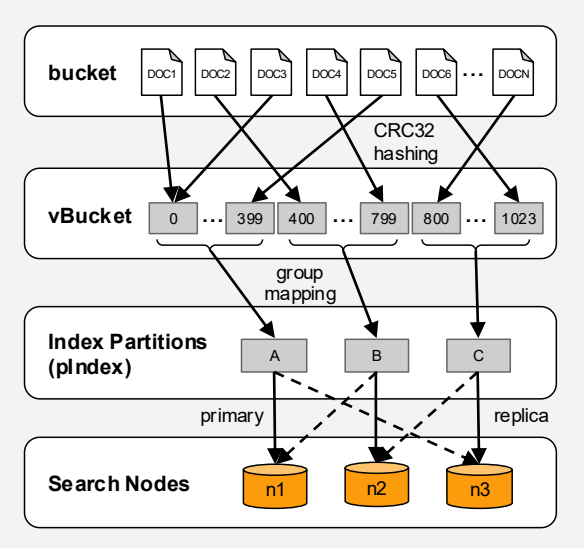
Inherited Features

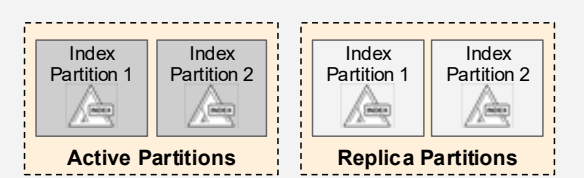
- Continuous Ingestion
- Upsert/Delete support
- Index Partitioning
- Distributed Execution
- Workload Isolation
- Independent Scaling
- High Availability

Plus (for Vector)

- Automatic Algorithm Selection
- Automatic Tuning
- Just "one dial" recall or latency or memory_efficient

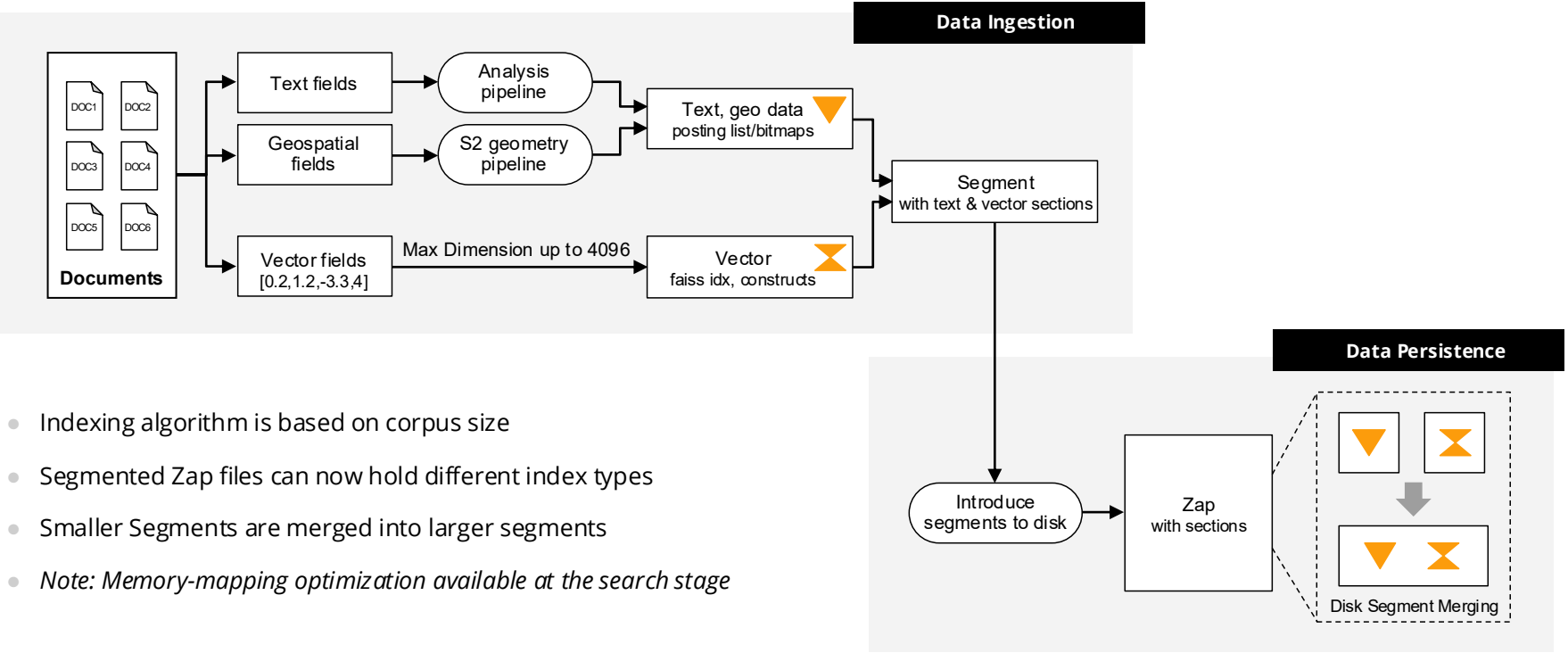






Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Concepts | Data Ingestion and Persistence Pipeline

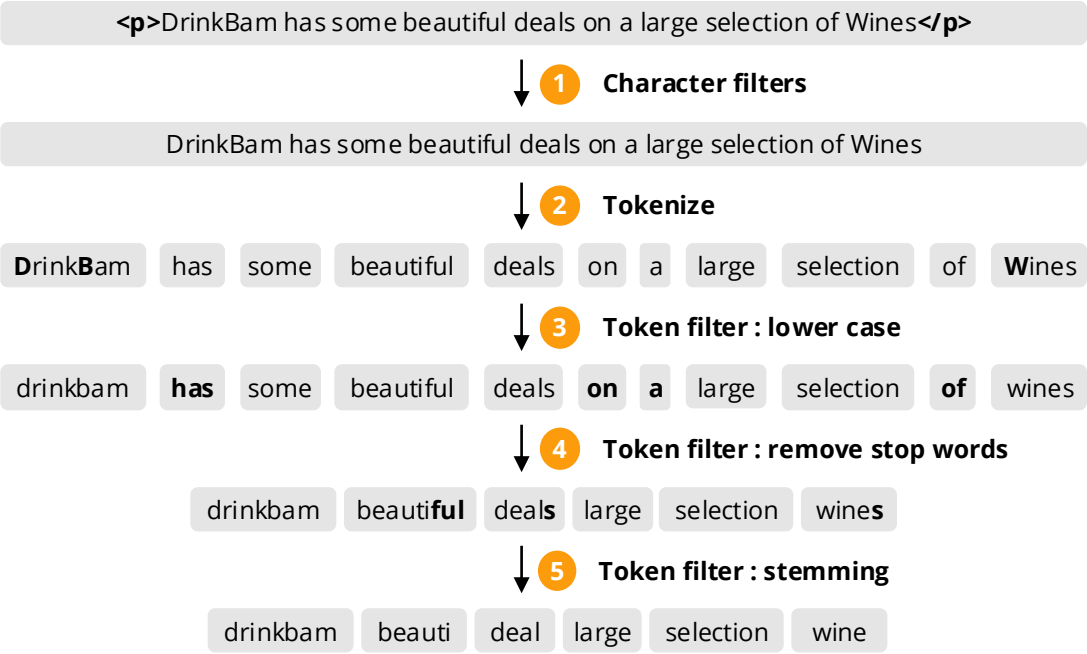


- Indexing algorithm is based on corpus size
- Segmented Zap files can now hold different index types
- Smaller Segments are merged into larger segments
- *Note: Memory-mapping optimization available at the search stage*

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Concepts | Analyzers

Transforming input text into analyzed tokens — getting rid of grammatical inflections while preserving the words' meaning.



- **Character Filters**: Remove undesirable characters or tags
- **Tokenizer**: Splits text streams into a stream of individual tokens
- **Token Filter — lower case**: Filters the individual tokens to lowercase
- **Token Filter — remove stop words**: Removes all the tokens associated with stop words
- **Token Filter — stemming :** Reduce words to their root or base form

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Concepts | Analyzers Language Support

Support for 28 languages is included

Name	Language
ar	Arabic
bg	Bulgarian
ca	Catalan
cjk	Chinese Japanese Korean
ckb	Kurdish
da	Danish
de	German
el	Greek

Name	Language
en	English
es	Spanish (Castilian)
eu	Basque
fa	Persian
fi	Finnish
fr	French
ga	Gaelic
gl	Spanish (Galician)
hi	Hindi
hu	Hungarian

Name	Language
hy	Armenian
id, in	Indonesian
it	Italian
nl	Dutch
no	Norwegian
pt	Portuguese
ro	Romanian
ru	Russian
sv	Swedish
tr	Turkish

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Concepts | Full inverted index

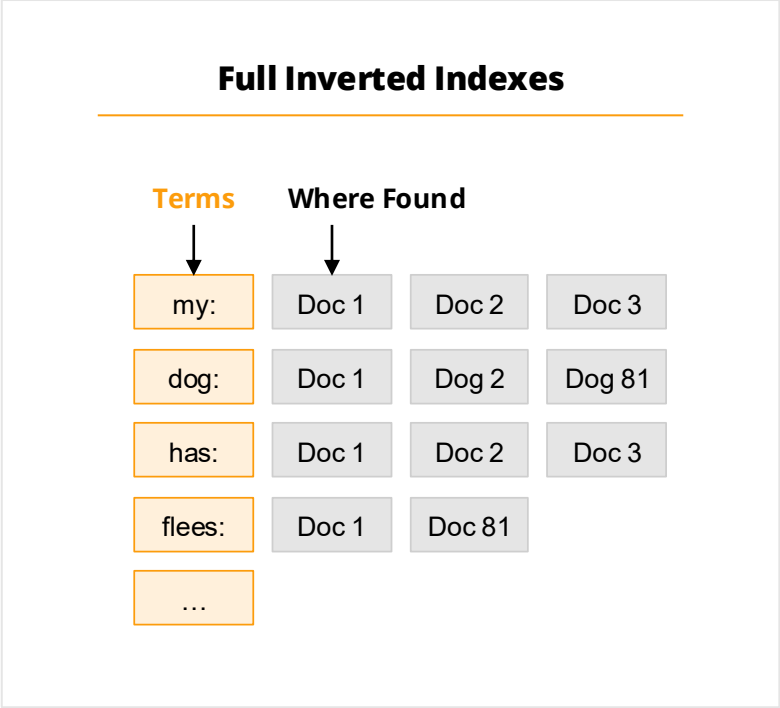
Storing and indexing tokens

For every analyzed token, the FTS index stores:

- Document Id
- Attribute chain
- Offset information
- Snippet (optional)

Unlike global secondary (GSI) indexes:

- No compound indices — this is a flat index
- Order of tokens in query is not important
- Value is not stored repeatedly in the index (unless snippets are stored), but Document ID and some metadata is repeated more often than GSI



Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Concepts | TF-IDF Scoring

Term Frequency-Inverse Document Frequency

Term frequency

- How often does a term occur in a document?
- More often yields a higher score

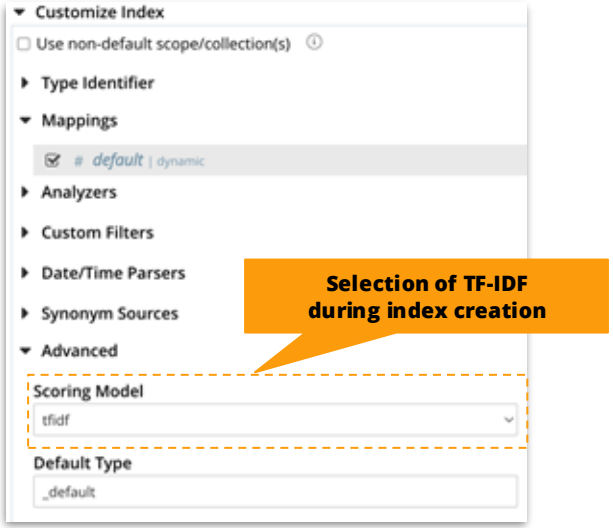
Inverse Document Frequency

- Importance & uniqueness of a term across a collection of documents

$$\text{TF-IDF}(t,d) = \left(\frac{\text{Occurrences of term } t \text{ in document } d}{\text{Total terms in document } d}\right) \times \log\left(\frac{N}{1+\text{DF}(t)}\right)$$

- N = Total number of documents in the corpus.
- DF(t) = Number of documents containing term t.

Selecting TF-IDF



Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Concepts | BM25 Scoring

From 8.0 version

Best Match 25

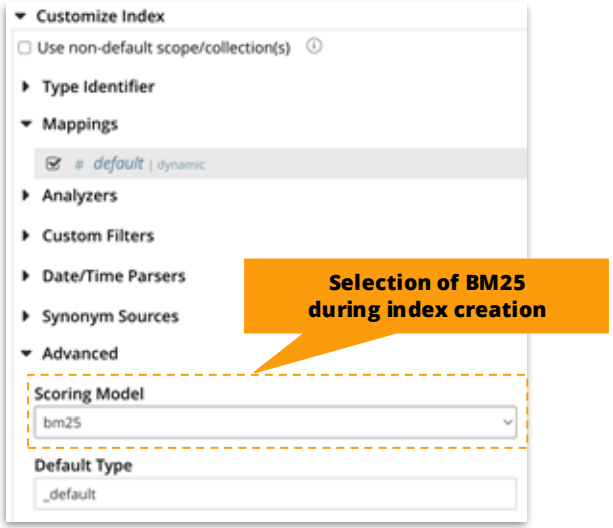
BM25 and TF-IDF comparison

- Both methods score words according to how "important" they are in the context of a document collection.
- BM25 is enhanced tf-idf which considers more metrics to deliver richer scores.

Using BM25

- Opt-in feature enabled during index definition.
- Query time control to request BM25 scores be computed at an index level, as opposed to at partition level - this keeps the scoring uniform regardless of the number of partitions in the index.
- BM25 scores will seamlessly integrate with kNN distances for hybrid search scenarios.

Selecting BM25



Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Concept | Search replica partitions

From 8.0 version

Reading replicas for Performance Improvement

Overview

- By default search is allowed only over active index partitions.
- New opt-in feature at query time to be able to search replica partitions via these control parameters:

["", "local", "local_only", random", "random_balanced"]

Performance Improvement

- Can help with performance at times by reducing network scatter-gather, and users can leverage this to restrict requests to server-groups/AZs.
- Ability to leverage this feature is being added into SDK.

Controlling Search Replica Partitions

Parameter	Description
""	Default - Queries target only active partitions
"local"	Prefers local active/replica partitions, then server group, then random node.
"local_only"	Prefers local partitions; if unavailable, returns partial results with a missing partition indicator.
"random"	Selects a random node with the active/replica partition.
"random_balanced"	Prefers nodes not already handling requests; falls back to "random" if needed

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Key Takeaways

Couchbase Search | Summary

Flexible, Language-Aware Search	SQL++ Integration & Flex Indexing	Search at Scale & High availability	Hybrid Search (Keyword + Vector)
• Matches natural language with relevance scoring on JSON data. • Rich queries: match, phrase, fuzzy, prefix, regex, wildcard & range • Geospatial search via geopoint or GeoJSON shapes.	• Combine FTS with SQL++ in one query (joins, filters + full-text). • Flex Index: run SQL++ as a full-text search using an FTS index. • SEARCH functions can also use XATTR metadata.	• Search indexes are partitioned and distributed across Search nodes (auto-balanced on rebalance). • Default partitioning guidance (e.g., “default number… is six”). • Failover behavior: replicas used or partitions rebuilt	• Couchbase search supports Vector Search alongside traditional text search. • Hybrid via REST/UI combines vector (knn) and text query as OR; add boost to balance TF-IDF vs KNN distance. • With SQL++, you can choose AND or OR for hybrid results, filter fields (e.g., numeric ranges) alongside vectors.

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Demystifying Vector Search

What is a Vector

This is a vector

2.6	11.3	-4.2
First value	Second value	Third value

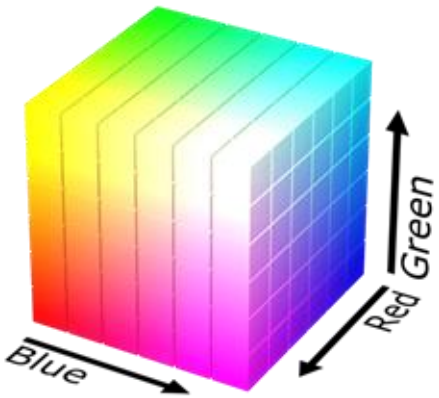
Here, it contains 3 values
=> its dimension is 3

A Vector is a just an **array of numerical values**

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

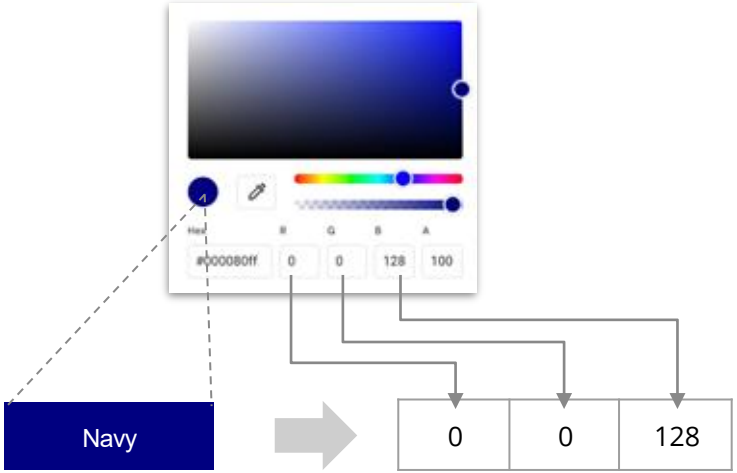
What is a Vector used for | Basic Color Example

The RGB model example



The model used to create the colors you see on TV and computer screens.
Each color is the addition of a **R**ed, **G**reen and **B**lue primary colors.

A Vector representation of a Color



The color *Navy* ...

... is represented in the RGB model with this vector

A **Vector** is used to **represent** a more complex object. The **Model** defines the **meaning** of each dimension.

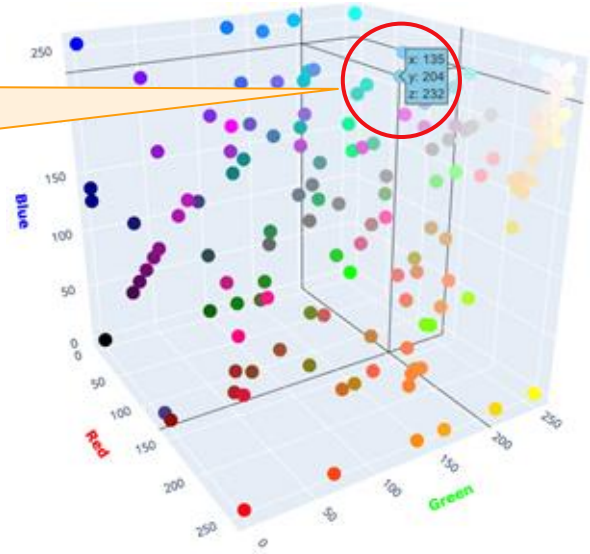
Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

What is Vector Search | Searching Similar Colors Example

Example of 123 vectors of RGB colors

Which are the colors similar to this one?

[135,204,232]



Top Results

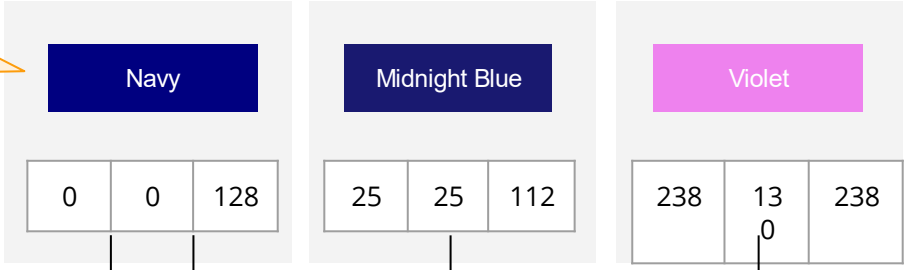
#1	sky blue	[135,206,235]
#2	light sky blue	[135,206,250]
#3	light blue	[173,216,230]

A vector search is a query that **finds vectors similar to the one you provide**

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

How does Similarity Works | Comparing Colors Example

To the human eyes,
Navy is closer to Midnight Blue than Violet



distance ≃ 38

distance ≃ 292

Mathematically,
we got the same result by comparing the vectors

Vectors are compared using **a similarity distance.**

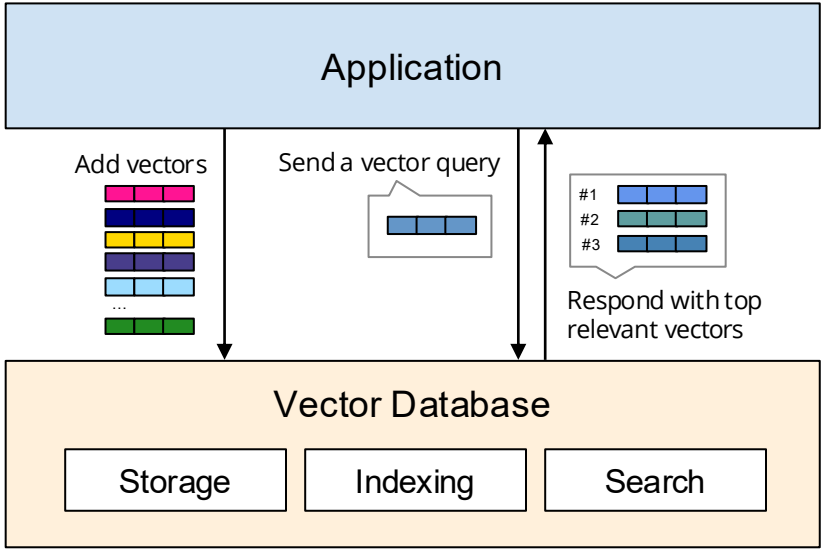
Here the *euclidean distance*

$292 \simeq \sqrt{(238-0)^2+(130-0)^2+(238-128)^2}$

Vectors can easily be compared mathematically using a **similarity distance**

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

What is Vector Database



Vector databases provide the ability to **store, index and search vectors**

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Couchbase Vector Storage

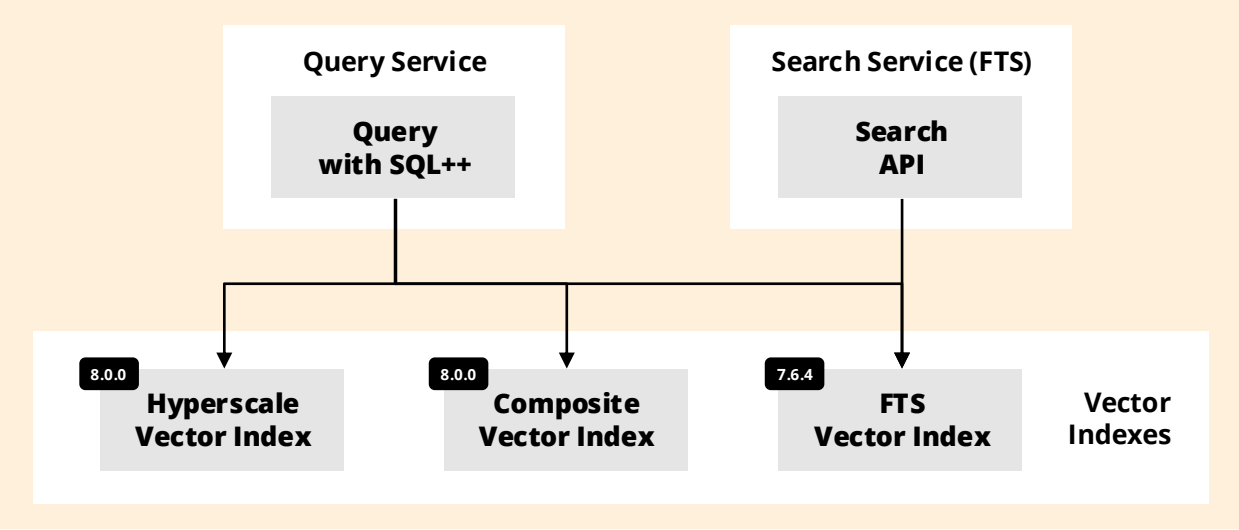
Data Service

```
{
 "id": "#000080",
 "color": "navy",
 "brightness": 14.592,
 "colorvect_l2": [0, 0, 128],
 "description": "Navy is a deep, rich color that exudes sophistication. It is
    a dark shade of blue that is often associated with authority, stability,
    and elegance. Navy is a versatile color that can be both bold and
    understated, making it a popular choice in fashion and interior design.
    It is a timeless color that never goes out of style and adds a touch of
    sophistication to any look or space.",
 }
```

Couchbase **stores vectors in JSON** Documents, **alongside** the other information

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Couchbase Vector Indexing and Search

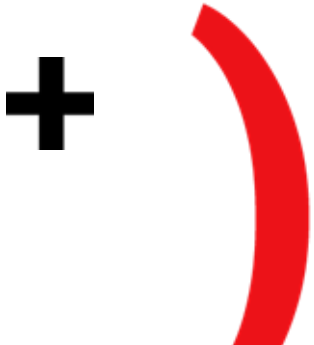


Couchbase provides **flexible and scalable ways** to perform Vector Search on your data

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

Vector Search Use Case:

Semantic Search



Where Is Semantic Search Used Today?

Recommendations

Semantic search enhances **product recommendations** by understanding customer preferences beyond keyword matches.

Content Discovery

Media platforms can leverage semantic search to help users discover **relevant articles, videos, or music**.

Fraud Detection

Understand customer behavior over time and **find patterns similar to past fraud**—without relying on exact matches.

Medical Diagnosis

In the medical field, semantic search assists doctors in **diagnosing diseases** and finding relevant research.

Customers want to build applications leveraging **Semantic Search on their own real-life private data**

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

How Can a Machine Understand Real-Life Things?

image

[0.9, 0.7, 0.1,...]

text

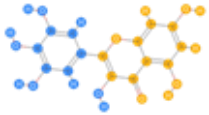
[0.2, 0.7, 0.3,...]

audio

[0.9, 0.8, 0.2,...]

video

[0.7, 0.9, 0.3,...]



molecule

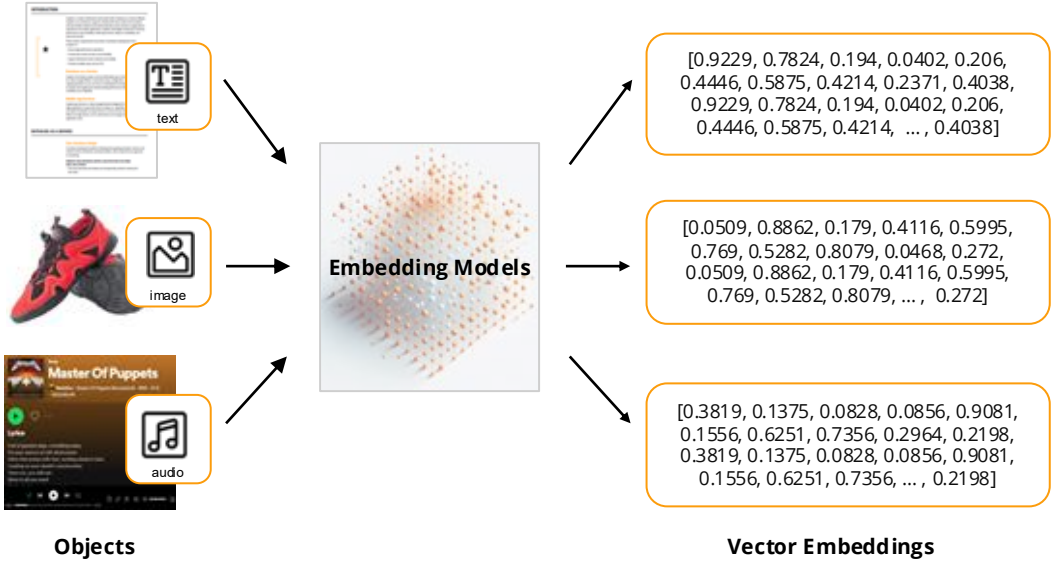
[0.4, 0.1, 0.8,...]

...

Real Life objects **must be converted into Vectors** so computers can compare them

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

How Are Real-Life Things Turned Into Vectors?

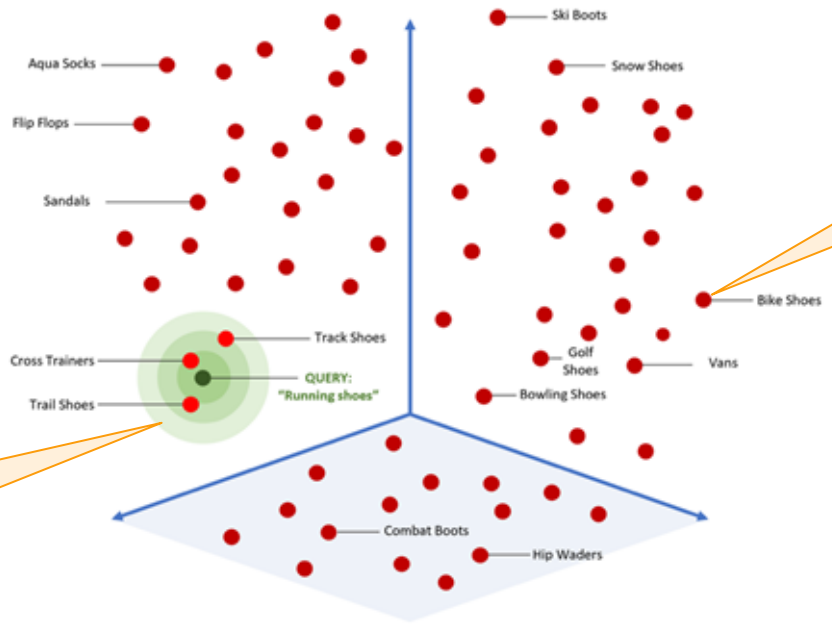


- Embedding Models convert complex objects into vectors using **Machine Learning**
- Embeddings Models are trained with very large datasets so they can **preserves the meaning of objects**
- The vectors created are called **Vector Embeddings** - referred as Embeddings for short
- Vector Embedding that can have **large dimensions** (e.g. 1536)

Embeddings models are algorithms trained to **convert objects into Vectors** (aka. Embeddings)

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

How Semantic Search Works with Text



Each dot is a vector representing the meaning of the related item

Similar items are represented by vectors close to each other

Search for matches based on the **MEANING** of the words, not just the words themselves

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Why Isn't Vector Search Enough on Its Own?

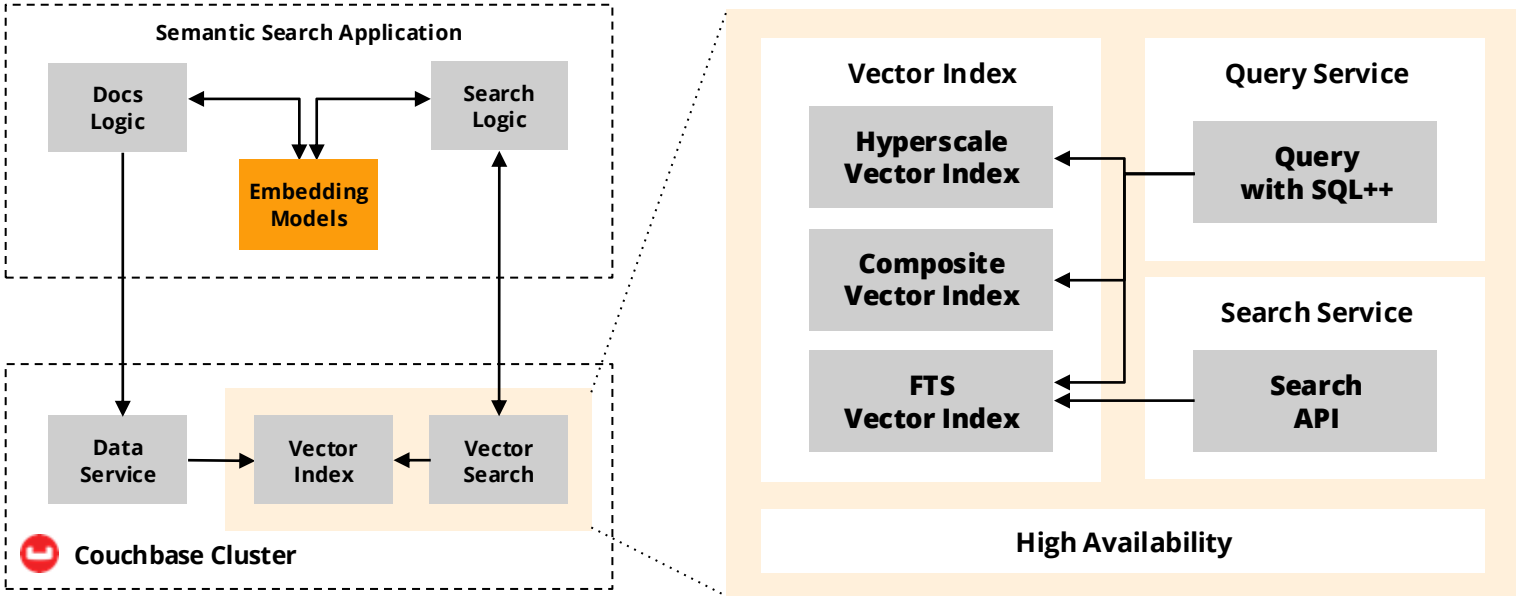
Complex user search prompt:
"Find casual and fun shoes that match the color of my car, between $100 and $200 with good reviews, in stock right now near me"

- Color to match an image of a car **(semantic/vector)**
- Description to mention "casual and fun" **(text/fuzzy)**
- Price between $100 and $200 **(range)**
- With rating over 4.5 stars **(range)**
- Within 15 miles **(geospatial)**
- Available today in stores **(exact match)**

Hybrid Search allows to build real-world **hyper-personalization use cases**

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

How does Couchbase support Semantic Search?



Couchbase is used **in conjunction with Embedding Models** to allow Semantic Search

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2024. All rights reserved.

Where Do Embedding Vectors Come From (in the Cloud)?

Vendor	Text Embeddings	Image Embeddings	Audio Embeddings	Multimodal
Microsoft (via Azure AI)	text-embedding-ada-002, Cohere embed-v3	Azure Vision, CLIP	Whisper, Azure Speech	GPT-4o (text+image+audio)
Google (via Vertex AI)	textembedding-gecko, PaLM-2/3	ViT, PaLI, Imagen	VATT, AudioLM	Gemini, PaLI-X
Meta (self-hosted)	❌ Not available	DINOv2, Segment Anything, SAM	Wav2Vec 2.0, HuBERT	Flamingo
Open Source (mostly self-hosted)	SBERT (all-MiniLM-L6-v2), BGE, GTR…	CLIP, BLIP, DINOv2, ResNet	Wav2Vec 2.0, CLAP, HuBERT	LLaVA, mPLUG-Owl2, OpenFlamingo

Powerful, multi-modal models delivered **via cloud APIs or self-hosted in the cloud** — built for scale and versatility.

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Can Semantic Search Run Offline on Mobile?

SQL++, full-text search & vector search for AI

Offline first design & peer-to-peer sync

Advanced data sync & user management

With Couchbase Mobile, Semantic Search Runs **Directly on the Device—Even Offline.**

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

What Kinds of Models Power Semantic Search on Devices?

Small language models *e.g.TinyBERT*, *MiniLM*, and *DistilBERT*	✅ Power fast, offline **text** understanding.
Vision-language models *e.g. BLIP-lite or LLaVA-mini*	✅ Help apps understand both **images and text**.
Quantized models *Specially compressed versions*	✅ **Run efficiently** on phones, sensors, and smart devices.

Small, efficient models optimized to run **directly on devices—no cloud required.**

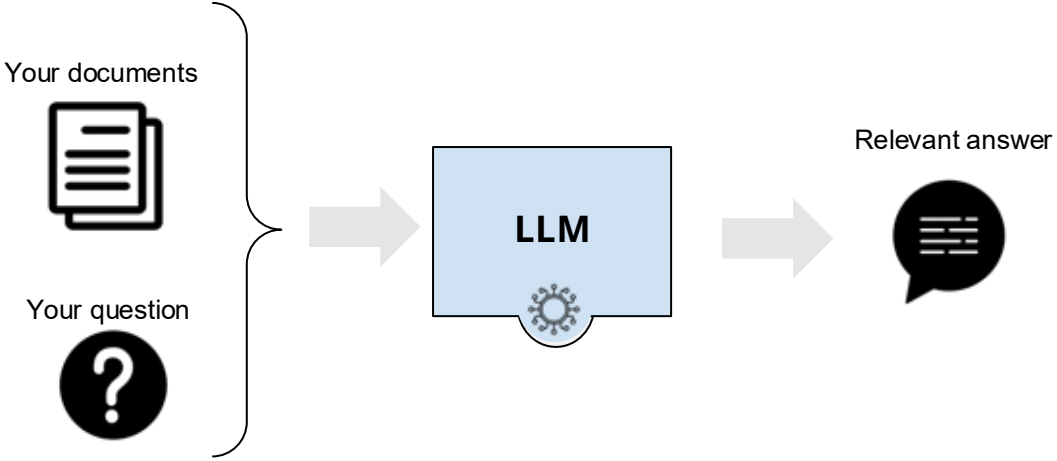
Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Vector Search Use Case:

Retrieval-Augmented Generation (RAG)

How Can You Chat with Your Own Data?

LLMs (e.g. ChatGPT, Gemini, Claude) **don't know** anything about your **private data**



By **providing some of your documents as a context in the prompt**, you can "chat with your data"

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

What Are the Risks of Sending Data to AI Models?

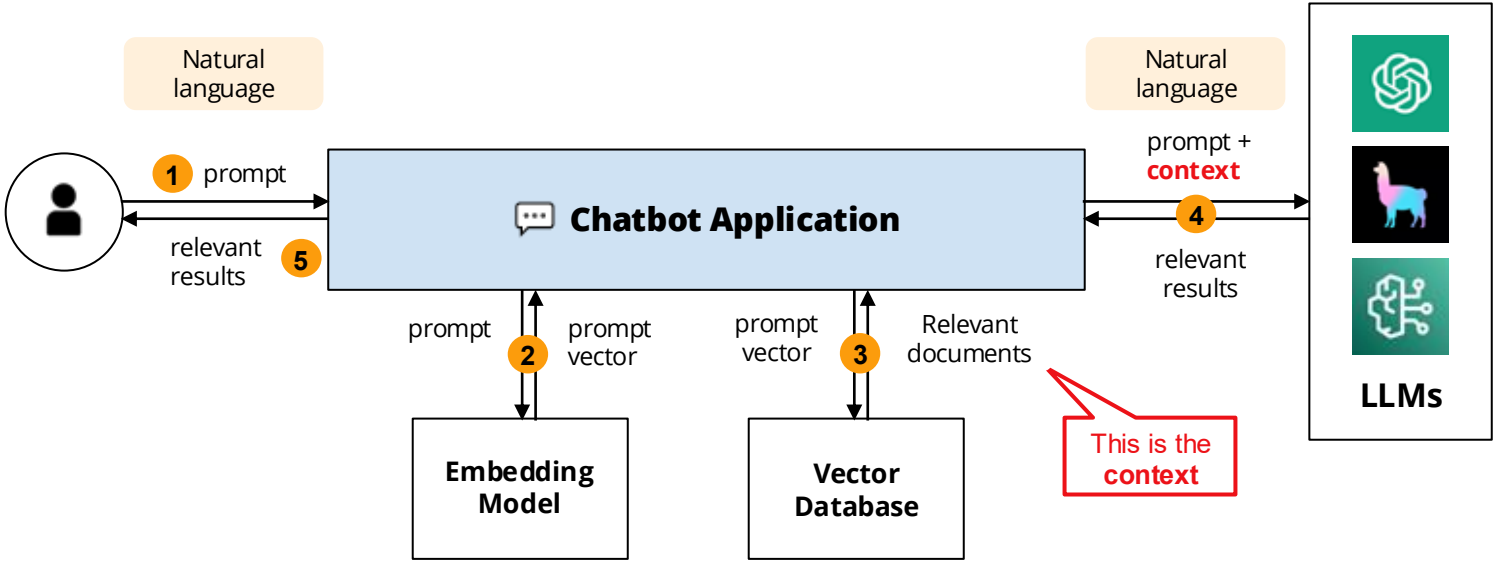
Sharing proprietary and sensitive data

Sharing data that induces hallucinations

These are C-level **showstoppers** if they are not addressed

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

How Does RAG Actually Work?



RAG **reduces the risk of LLM hallucinations** by providing a **relevant context** in the prompt.

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

What Makes RAG So Powerful?

Reduce Hallucinations

RAG adds **trusted context** to the prompt—helping the LLM generate more accurate, grounded responses.

Controlled Access to Data

Developers can apply **access rules and filters** at the retrieval layer—so only approved content is sent to the LLM.

Cost-Effective Solution

Cheaper to introduce new, domain-specific data without retraining the LLM.

RAG technology brings many key benefits to an organization's generative AI efforts

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

How Do I Build RAG Apps Faster?

LangChain

Leverage LangChain for document loading, vector storage, semantic caching

Langflow

Build RAG & multi-agent apps with Couchbase and Langflow

LlamaIndex

Enhance document storage & search with vector embeddings and efficient indexing

Amazon Bedrock

Create AI chat apps with AWS Bedrock & Couchbase

haystack by deepset

Combine Haystack & Couchbase for indexing and dense retrieval

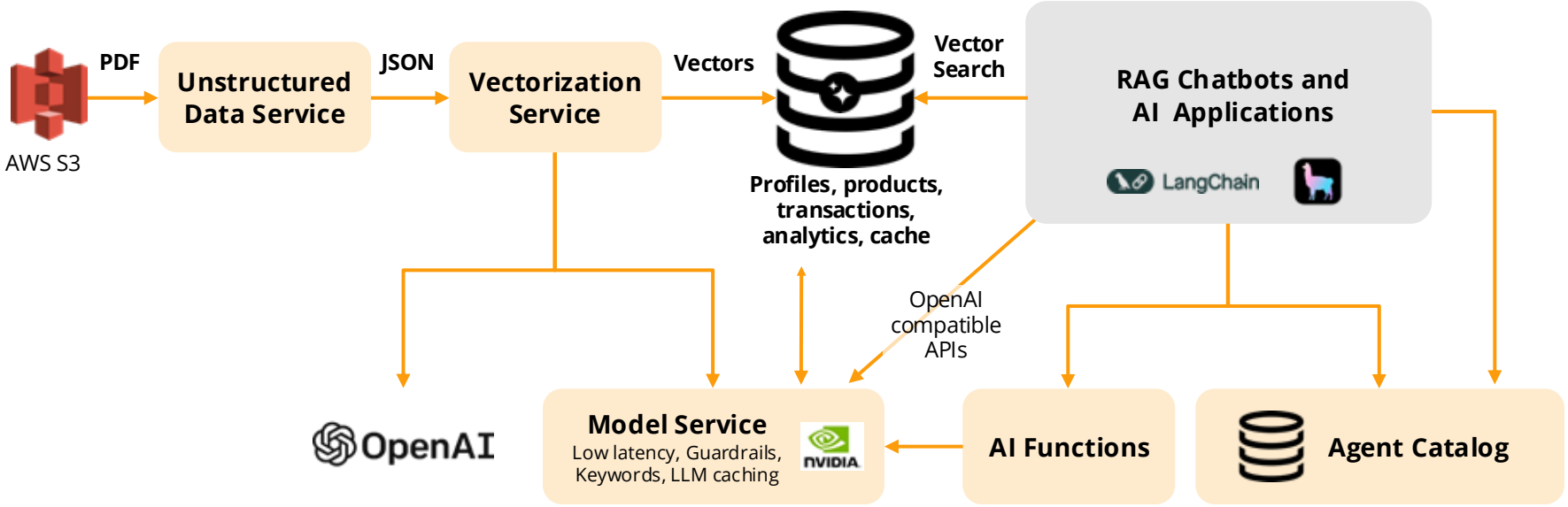
mindsdb

Integrate AI/ML with Couchbase using turnkey MindsDB

Couchbase constantly expand its ecosystem integration to make it **easier to build smart, scalable apps**

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Build AI & RAG Apps with Capella—No Extra Tools Needed



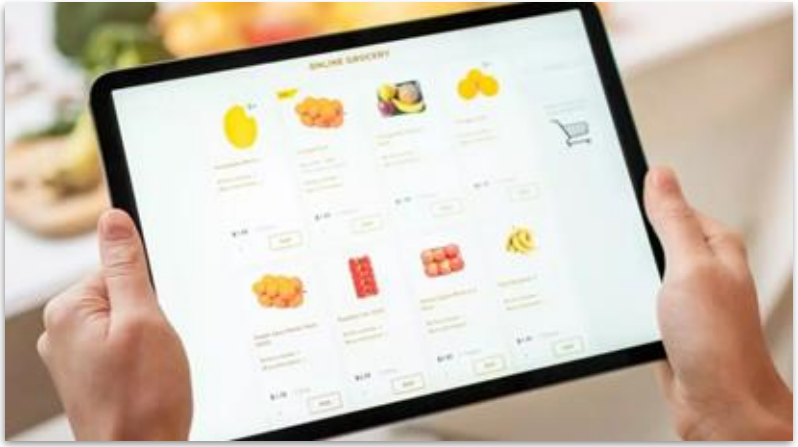
Couchbase Capella is the **Developer Data Platform** designed to be a holistic data solution **for AI applications**

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Key Takeaways

Real-World Use Cases: Semantic Search & RAG in Action

Advanced Semantic / Hybrid Search

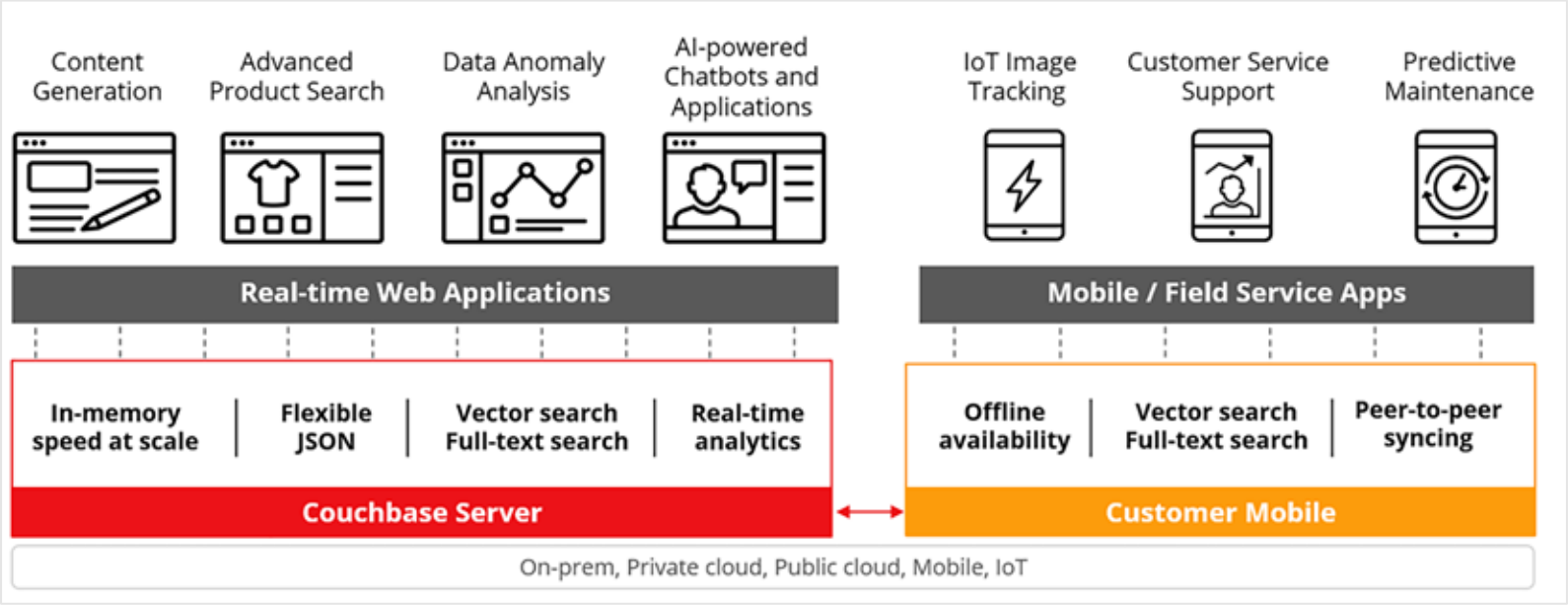


RAG Chatbots and AI-powered Apps

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Vector Search Across our Products

First in the industry to support for all 3 deployments: Cloud, on-prem, mobile



Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

AI Agents Are Transforming Industries

AI agents are tackling specialized tasks within various industries

Banking

AI agents are streamlining operations with automated contract analysis and real-time fraud detection, reducing manual overhead and time

Retail

AI agents are transforming customer experiences through personalized shopping and optimizing backend logistics like inventory and restocking

Healthcare

AI agents assist clinicians with intake triage, documentation, and diagnosis support, improving care efficiency and reducing administrative burden

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Why Agentic AI Needs Vector Search

Agentic AI applications need to support

Billion scale vector search

Scaling to billion scale vector datasets with millisecond latency

High dimensional vectors

Supporting higher dimensional vectors for richer data representation

Vector search across modalities

Expanding to multi modal search across text, image, and video

Hybrid search capabilities

Combining vector, filters and keyword search for hybrid retrieval capabilities

***Goal*: Deliver fast, accurate, and flexible search across massive, multi-modal datasets using high-dimensional representations**

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

Why Couchbase Is Built for This Moment

User Applications and Agents

Agentic AI *RAG Chatbots* *Predictive ML* *Product Catalog* *Operational Analytics/BI*
Field Service Apps *IoT* *Customer 360* *Profiles & Personalization* *Caching & Session Management*

Other Data Interactions

Connectors

Semistructured

Unstructured

Multi Source

Data Access (SDKs, SQL+, MCP, Data API, Natural Language)

JSON

Transactional	**Mobile Edge**	**Analytical**	**AI**
Key-value cache & persistence	*Sync & offline mobile persistence*	*Columnar persistence*	*Billion scale vector index*

Enterprise-grade Security (SOC 2, HIPAA, GDPR, PCI DSS, ISO 27001)

AI Integrations

Model Platforms

Frameworks & Tools

Security & Observability

Data Catalog & Governance

ENTERPRISE (Customer Managed)

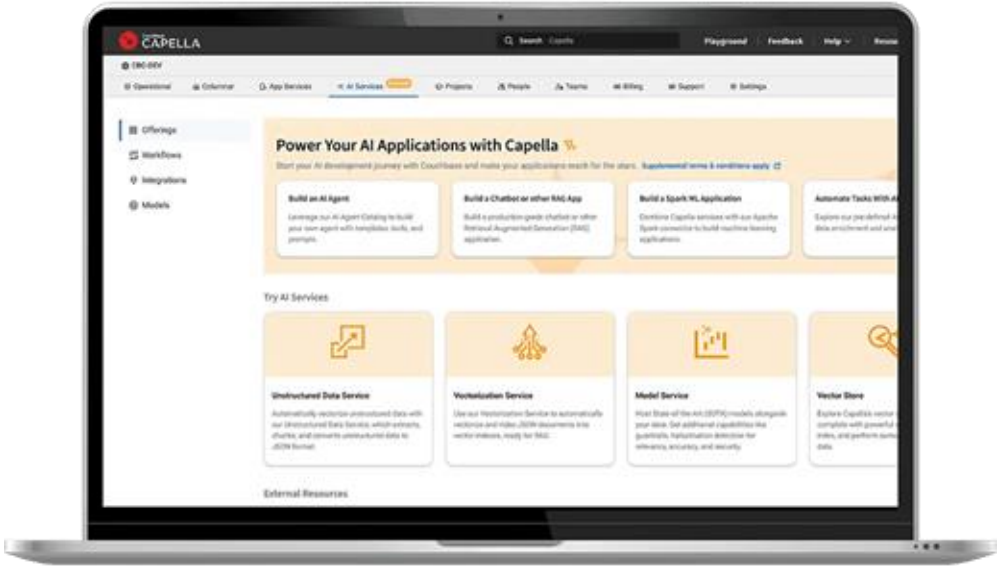
Public Cloud Cloud Edge On Prem

CAPELLA (As-a-Service)

Public Cloud

Confidential and Proprietary. Do not distribute without Couchbase consent. © Couchbase 2026. All rights reserved.

THANK YOU!



Couchbase